PROYECTO BIBLIOTECA IUB

Documentación académica

Estudiante: Isaacs Ferrer
Programa: Ingeniería Telemática
Institución: Institución Universitaria de Barranquilla (IUB)
Asignatura: [Nombre de la asignatura]
Docente: [Nombre del docente]
Fecha de elaboración: 10 de octubre de 2026

Tabla de contenido

Tabla de contenido 2
Introducción 3
Planteamiento del problema 3
Justificación 3
Objetivo general 3
Objetivos específicos 3
Población beneficiaria 4
Alcance y limitaciones 4
Requisitos funcionales 4
Requisitos no funcionales 5
Descripción de los módulos administrativos y de usuario 6
Módulos administrativos 6
Módulos de usuario 7
Arquitectura del sistema 7
Tecnologías utilizadas 8
Modelo de datos 8
Autenticación, autorización, roles y modo demostración 9
Metodología de desarrollo 9
Plan de pruebas y resultados comprobados 9
Criterios de aceptación 10
Limitaciones y trabajo pendiente 10
Conclusiones 10
Referencias técnicas oficiales 10

Introducción

El presente documento corresponde a la documentación académica del proyecto “Biblioteca IUB”, desarrollado en el marco del programa de Ingeniería Telemática. El proyecto consiste en un sistema web para la gestión de acceso y préstamos de recursos bibliográficos e informáticos de la Institución Universitaria de Barranquilla (IUB). El sistema combina un backend FastAPI con base de datos PostgreSQL y un frontend Angular, e incorpora autenticación por roles (JWT), un modelo de permisos (RBAC), gestión de libros, dispositivos y préstamos, servicios de inteligencia artificial y un modo demostración independiente de los datos reales.

Todo el contenido de este documento se elaboró con base en información verificable del repositorio: estructura del código, configuración de despliegue, modelo de datos y pruebas automatizadas. No se incluyen ítems ficticios ni resultados no comprobados.

Planteamiento del problema

La biblioteca de la IUB presta servicios a estudiantes, docentes y funcionarios en las sedes de Soledad, Barranquilla y Colsamiro. El control de préstamos de libros y de dispositivos (computadores y tabletas), así como el registro de ingresos, se lleva mediante procesos que requieren trazabilidad, control de identidad y aplicación de reglas de negocio (plazos de 4 a 6 días para libros y franjas de máximo 2 horas para dispositivos). Sin una plataforma única se dificulta conocer la disponibilidad real de los recursos, validar la identidad de quien solicita un recurso y restringir cada usuario a las funciones propias de su rol.

El proyecto aborda esta problemática mediante un sistema centralizado con autenticación por roles y respaldo de datos en PostgreSQL, de modo que cada usuario solo pueda ejecutar las operaciones que su rol le permite.

Justificación

Una plataforma centralizada reduce la pérdida de información, agiliza el préstamo y la devolución de recursos y deja evidencia de cada operación. La autenticación con JWT y el modelo de permisos por rol garantizan la separación de funciones: los estudiantes solicitan recursos, los administradores gestionan inventario y aprueban solicitudes, y el super administrador administra usuarios y privilegios. Los servicios de búsqueda y recomendación basados en técnicas de inteligencia artificial mejoran la experiencia del usuario, y el modo demostración permite presentar el sistema sin depender de credenciales ni datos reales.

Objetivo general

Desarrollar un sistema integral para la gestión de acceso y préstamos de recursos bibliográficos e informáticos de la Biblioteca IUB, con autenticación basada en roles, módulos administrativos y de usuario, servicios de inteligencia artificial y un modo demostración independiente, respaldado por PostgreSQL.

Objetivos específicos

1. Implementar un backend con FastAPI y PostgreSQL que incluya autenticación JWT, hashing de contraseñas con bcrypt y un modelo de permisos por rol (RBAC) con revalidación del rol desde la base de datos en cada petición protegida.

2. Construir un frontend con Angular que ofrezca páginas por rol, guards de navegación, formularios validados y servicios centralizados sobre una única configuración de URL (environment).

3. Restringir el ingreso de modo que el rol seleccionado en el login coincida con el rol real de la cuenta.

4. Incorporar un modo demostración separado de la autenticación real, sin persistencia y sin llamadas a la API.

5. Integrar servicios de inteligencia artificial para búsqueda, recomendación y estadísticas, y el registro de accesos por código QR.

6. Configurar el despliegue del backend en Render y la base de datos en Neon, con una gestión de conexiones adecuada para un servicio gestionado.

Población beneficiaria

Estudiantes, docentes y funcionarios de la Institución Universitaria de Barranquilla que utilizan los servicios de la biblioteca en las sedes de Soledad, Barranquilla y Colsamiro. El super administrador y los administradores operan el sistema; los docentes, funcionarios y estudiantes acceden de acuerdo con sus permisos.

Alcance y limitaciones

Dentro del alcance: autenticación y autorización por roles; gestión de usuarios, libros, dispositivos y préstamos; aprobación y devolución de solicitudes; catálogo y búsqueda (tradicional y semántica); recomendaciones; panel estadístico; registro de ingresos por DNI; exportación de reportes; modo demostración; y configuración de despliegue del backend en Render con base de datos en Neon.

Limitaciones: el frontend aún no consume la API Express/MySQL de proveedores y compras (aunque esa API existe en el repositorio); el frontend no está desplegado en un hosting; ciertas pruebas del backend y las pruebas unitarias del frontend están pendientes; y la validación funcional en vivo contra el despliegue de Render y Neon está pendiente de confirmación.

Requisitos funcionales

ID	Módulo	Requisito funcional
RF-01	Autenticación	Iniciar sesión con identificación y contraseña; la contraseña se valida con bcrypt.
RF-02	Autenticación	Emitir un token JWT (HS256) con vigencia de 8 horas y el rol del usuario desde la base de datos.
RF-03	Autenticación	Validar que el rol seleccionado en el formulario coincida con el rol real de la cuenta; bloquear el ingreso si no coincide.
RF-04	Autenticación	Cerrar sesión eliminando el token, los datos de sesión y desactivando el modo demostración.

ID	Módulo	Requisito funcional
RF-05	Usuarios	Crear, listar, actualizar y eliminar usuarios; restringir la creación de administradores al super administrador.
RF-06	Libros	Listar libros (por sede), buscar, crear, actualizar y eliminar.
RF-07	Dispositivos	Listar, crear, actualizar y eliminar dispositivos (computadores y tabletas).
RF-08	Préstamos de libros	Registrar préstamos con plazo de 4 a 6 días y estado de solicitud.
RF-09	Préstamos de libros	Aprobar, rechazar y devolver préstamos; descontar y devolver el inventario según corresponda.
RF-10	Préstamos de libros	Mostrar a cada usuario únicamente sus propios préstamos; el administrador ve todos.
RF-11	Préstamos de dispositivos	Solicitar un dispositivo para sí mismo con franja máxima de 2 horas entre las 8:00 y las 20:00.
RF-12	Préstamos de dispositivos	Gestionar solicitudes (aprobar, rechazar, completar y marcar daño).
RF-13	Catálogo	Mostrar catálogo con categorías y búsqueda semántica (IA).
RF-14	Chatbot	Responder consultas sobre libros, préstamos y reglamento mediante plantillas o LLM opcional.
RF-15	Recomendaciones	Generar recomendaciones según el perfil académico del usuario.
RF-16	Dashboard	Mostrar indicadores: tendencia, libros más prestados, disponibilidad crítica, morosidad y demanda.
RF-17	Ingresos	Registrar accesos a la biblioteca por DNI (kiosco QR) y visualizar los ingresos con totales.
RF-18	Reportes	Exportar reportes de libros, usuarios, dispositivos, préstamos e ingresos (PDF y Excel).
RF-19	Modo demostración	Navegar por las pantallas con datos de ejemplo sin llamar a la API ni usar JWT; no debe restaurarse al recargar.
RF-20	Seguridad de rutas	Proteger las rutas del frontend por rol (guards) y las operaciones del backend por módulo (RBAC).

Requisitos no funcionales

Categoría	ID	Requisito no funcional
Seguridad	RNF-01	Las contraseñas se almacenan con bcrypt; nunca en texto plano para usuarios nuevos.
Seguridad	RNF-02	Los tokens JWT se firman con HS256; el rol se revalida desde la base de datos en cada petición protegida.
Seguridad	RNF-03	CORS restringido a los orígenes configurados por variable de entorno; no se usa comodín con credenciales.
Seguridad	RNF-04	Limitador de peticiones por IP en el login y en el registro de accesos QR.
Seguridad	RNF-05	Los mensajes de error no exponen contraseñas, tokens, IPs ni detalles técnicos.
Rendimiento	RNF-06	Pool de conexiones SQLAlchemy con pool_pre_ping, reciclado de 300 s y tamaño acotado, acorde a un servicio gestionado.
Rendimiento	RNF-07	Índices y claves foráneas definidas en el esquema para las consultas frecuentes.
Usabilidad	RNF-08	Interfaz responsive basada en Bootstrap 5, con tablas de datos en contenedores adaptables.
Usabilidad	RNF-09	Tablas con búsqueda, orden y paginación mediante DataTables.
Compatibilidad	RNF-10	Backend compatible con Python 3.11 (Render) y PostgreSQL; frontend compatible con navegadores modernos.
Mantenibilidad	RNF-11	Backend modular (modelos, routers y servicios) con nombres descriptivos y suites de pruebas automatizadas.
Mantenibilidad	RNF-12	La configuración de despliegue y las variables de entorno se documentan en archivos versionados (render.yaml) sin secretos.

Descripción de los módulos administrativos y de usuario

Módulos administrativos

Los módulos administrativos los utilizan los roles con permisos administrativos (super administrador, administrador y funcionario, según la matriz):

– Dashboard: panel con indicadores de actividad, tendencia y disponibilidad generados con técnicas de análisis y predicción.

– Gestión de usuarios: alta, edición, listado y eliminación de usuarios, con jerarquía de roles.

– Gestión de libros: administración del catálogo con filtros por sede y herramientas de búsqueda.

– Gestión de dispositivos: administración del inventario de computadores y tabletas y de sus préstamos.

– Gestión de préstamos: aprobación, rechazo, devolución y daño de préstamos de libros y dispositivos.

– Estadísticas: indicadores de accesos y dispositivos.

– Ingresos: registro y seguimiento de accesos a la biblioteca e inicio del lector de código QR.

Módulos de usuario

– Inicio: portal con recomendaciones para el usuario.

– Catálogo: consulta de libros con categorías y búsqueda semántica.

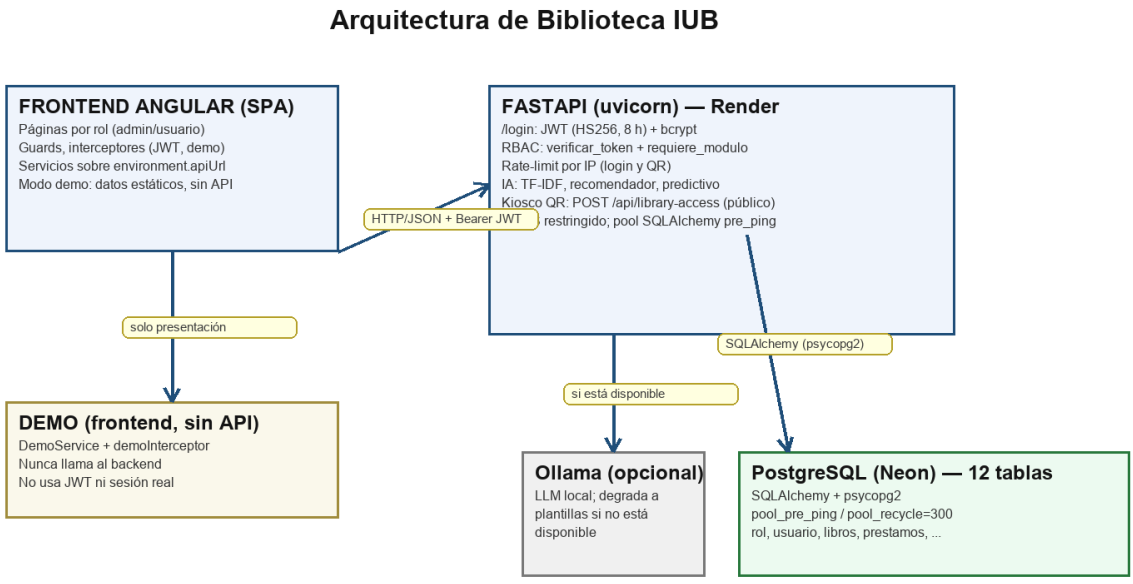
– Mis préstamos: consulta y solicitud de préstamos de libros (quedan pendientes de aprobación).

– Dispositivos: solicitud de computadores o tabletas con validación de franja y horario.

Arquitectura del sistema

El sistema usa una arquitectura de tres capas. El frontend Angular (SPA) consume una API REST de FastAPI mediante HTTP/JSON e incluye el token JWT en el encabezado Authorization. La API, desplegada en Render, aplica autenticación, autorización por módulo y reglas de negocio. La persistencia se realiza en PostgreSQL (Neon) a través de SQLAlchemy con psycopg2 y un pool configurado con pool_pre_ping para un servicio gestionado. Los servicios de inteligencia artificial (búsqueda semántica TF-IDF, recomendador y predicción) se ejecutan en el propio backend; el cliente LLM opcional (Ollama) degrada automáticamente a plantillas si no está disponible. El modo demostración es exclusivo del frontend e intercepta las peticiones sin contactar el backend.

Diagrama de la arquitectura (elementos verificados en el código):



Tecnologías utilizadas

Tecnología	Uso en el proyecto
Angular + TypeScript	Frontend (SPA, componentes standalone, señales, guards, interceptores).
Bootstrap 5 / Bootstrap Icons	Diseño responsive.
DataTables (jQuery)	Tablas con búsqueda, orden y paginación.
jsPDF / SheetJS	Exportación de reportes PDF y Excel en el cliente.
FastAPI + Uvicorn	API REST del backend.
SQLAlchemy	ORM y pool de conexiones.
psycopg2	Controlador de PostgreSQL.
python-jose	Generación y validación de tokens JWT.
passlib + bcrypt	Hashing de contraseñas.
scikit-learn / pandas	Búsqueda semántica, recomendación y predicción.
PostgreSQL	Base de datos.
Neon	Hosting de PostgreSQL.
Render	Hosting del backend (uvicorn).
Express + MySQL	API auxiliar de proveedores y compras (no integrada aún al frontend).

Las referencias oficiales de estas tecnologías se listan en la sección final; la bibliografía formal debe completarse según el formato institucional.

Modelo de datos

El modelo de datos declara 12 tablas que coinciden con el esquema “biblioteca_iub_postgres.sql” del repositorio:

Entidad	Propósito
rol	Roles del sistema (super administrador, Administrador, Funcionario, Docente, Estudiante).
usuario	Usuarios con identificación, contraseña (hash) y rol por clave foránea.
modulos	Catálogo de módulos de la aplicación.
modulos_X_rol	Matriz de permisos entre roles y módulos (RBAC).
libros	Catálogo de libros (autor, título, programa, ejemplares, sede).
prestamos	Préstamos de libros con estado y solicitud.
dispositivos	Inventario de computadores y tabletas por sede y estado.
prestamos_dispositivos	Solicitudes y gestión de préstamos de dispositivos.
library_access	Registro de accesos a la biblioteca (DNI, mensaje, estado).
contratistas	Proveedores (usados por la API Express auxiliar).
proveer	Relación entre libros y contratistas.
configuracion	Parámetros de configuración (moneda, horas, plazos).

Autenticación, autorización, roles y modo demostración

La autenticación ocurre en POST /login con identificación y contraseña. La contraseña se verifica con bcrypt y, si es correcta, se emite un JWT (HS256) con vigencia de 8 horas. El rol del usuario se toma de la base de datos. En cada petición protegida, la dependencia verificar_token valida la firma y la expiración y relee el rol actual desde la base de datos; si un administrador es degradado, su token deja de actuar como administrador. La autorización se aplica por módulo mediante la matriz modulos_X_rol (require_modulo). Las operaciones administrativas están limitadas a los roles con el módulo correspondiente; un estudiante obtiene respuestas 403 en esas rutas, aunque intente manipular el frontend.

El frontend replica la matriz de permisos únicamente para la navegación (guards y menús); el backend conserva la autoridad real. Además, el login valida que el rol seleccionado coincida con el rol real devuelto por el backend: si no coinciden, se cierra la sesión y no se navega.

El modo demostración permite recorrer las pantallas con datos de ejemplo sin credenciales y sin contactar la API. Se activa desde el login, se mantiene solo en memoria (no se restaura al recargar) y sus datos ficticios están separados de los datos reales. Este modo no cuenta como sesión autenticada: la autenticación real exige un JWT válido.

Metodología de desarrollo

El desarrollo se realizó mediante un proceso iterativo con integración y pruebas continuas. Las etapas principales fueron: (1) levantamiento de las funcionalidades requeridas y definición del modelo de datos sobre PostgreSQL (12 tablas); (2) diseño e implementación del backend con FastAPI, autenticación JWT, RBAC y servicios de inteligencia artificial; (3) implementación del frontend con Angular, guards, servicios centralizados y modo demostración; (4) automatización de pruebas del backend con pytest sobre una base SQLite aislada; (5) preparación del despliegue con Render y Neon, incluyendo la configuración del pool de conexiones; y (6) documentación técnica y académica. Se utilizó Git para el control de versiones y pruebas locales antes de cada integración.

Plan de pruebas y resultados comprobados

Prueba	Resultado
Suite de pruebas del backend (pytest, base SQLite aislada)	Reproducida en esta sesión: 100 pruebas aprobadas
Pruebas de autenticación, autorización y revalidación de rol	Aprobadas (parte de la suite)
Pruebas de préstamos, IA, RBAC, esquema PostgreSQL y pool de conexiones	Aprobadas (parte de la suite)
Compilación de producción de Angular (ng build --configuration production)	Reproducida en esta sesión: exitosa
Verificación de formato de cambios (git diff --check)	Reproducida en esta sesión: sin errores
Pruebas unitarias del frontend (ng test)	Pendiente: el proyecto no tiene configurado el objetivo test ni archivos .spec
Validación funcional en vivo contra Render y Neon	Pendiente de confirmación

Según la auditoría previa del repositorio, las 100 pruebas del backend cubren: autenticación y login (11), autorización del login (5), super administrador y RBAC (15), usuarios (9), libros y búsqueda (9), préstamos de libros y dispositivos (23), acceso QR (8), IA (10), pool y cierre de

sesiones (5) y esquema PostgreSQL base (5).

Criterios de aceptación

El sistema se considerará aceptado si: (1) el login valida credenciales reales y bloquea el acceso cuando el rol seleccionado no coincide con el rol de la cuenta; (2) las operaciones administrativas están protegidas por rol en el backend (403 para roles sin permiso); (3) los préstamos respetan plazos y franjas; (4) el modo demostración funciona sin datos reales ni llamadas a la API; (5) la suite de pruebas del backend pasa al momento de la presentación; y (6) la compilación de producción del frontend finaliza sin errores.

Limitaciones y trabajo pendiente

#	Trabajo pendiente
1	Validar en vivo el login y las rutas protegidas contra el backend de Render y la base de datos Neon.
2	Configurar las pruebas unitarias del frontend (ng test) y escribir casos para login, demo y guards.
3	Añadir pruebas del backend para el CRUD de libros y dispositivos y los estados Completado/Dañado de préstamos de dispositivos.
4	Definir el alcance de la API Express/MySQL (Requisito 2) y su eventual consumo desde el frontend.
5	Completar el README raíz del proyecto y, si aplica, la rúbrica del docente.
6	Eliminar código duplicado o sin uso y revisar la matriz de permisos del rol Funcionario.
7	Revisar aspectos de seguridad: enumeración de DNI en el registro público de accesos y el secreto JWT por defecto.
8	Confirmar el despliegue del frontend en su hosting y el commit pendiente de cambios menores de interfaz.

Conclusiones

El proyecto Biblioteca IUB ofrece una base sólida para la gestión bibliotecaria: autenticación con JWT y autorización por roles, módulos administrativos y de usuario, servicios de inteligencia artificial y un modo demostración separado. La evidencia disponible muestra 100 pruebas del backend aprobadas (reproducidas en esta sesión en el repositorio local), compilación de producción del frontend exitosa y un esquema de datos coherente con el código. Quedan pendientes la validación en vivo del despliegue, las pruebas del frontend, la limpieza de código y la documentación académica final; por ello no se afirma un cumplimiento del 100 %.

Referencias técnicas oficiales

Angular. Documentación oficial. https://angular.dev

Bootstrap. Componentes CSS y JavaScript. https://getbootstrap.com

FastAPI. Framework web para APIs. https://fastapi.tiangolo.com

SQLAlchemy. Kit de herramientas SQL y ORM para Python. https://www.sqlalchemy.org

psycopg2. Adaptador de PostgreSQL para Python. https://www.psycopg.org

PostgreSQL. Sistema de gestión de bases de datos. https://www.postgresql.org

Neon. Plataforma de PostgreSQL gestionado. https://neon.tech

Render. Plataforma de despliegue de aplicaciones. https://render.com

scikit-learn. Biblioteca de aprendizaje automático. https://scikit-learn.org

DataTables. Tablas de datos avanzadas. https://datatables.net

Elaborado por: Isaacs Ferrer – Ingeniería Telemática – Institución Universitaria de Barranquilla (IUB).